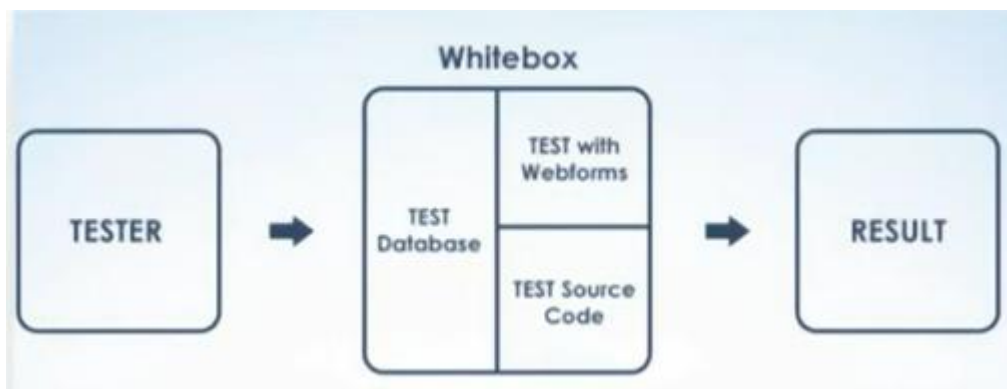


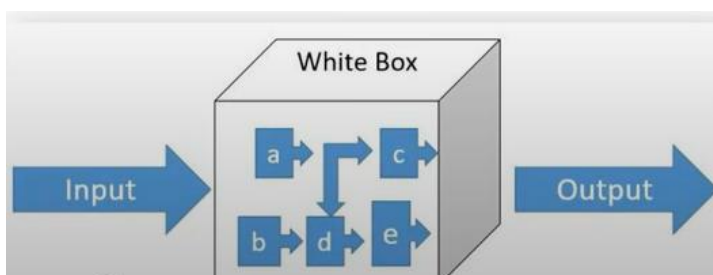
White Box Testing

- White box testing techniques analyze Internal design, Code structure & working of the complete software product.
- It is also called Clear box testing, Open box testing, Transparent box testing, Code-based testing and Glass box testing.
- Tester has knowledge about the internal structure or the programming of the software.
- It is mostly done by software developers.
- Perform with Unit Testing & Integration Testing.
- White Box Testing Tools: EcEmma, Nunit, PyUnit, HTMLUnit, CppUnit



- Internal security of code
- The flow of specific inputs through the code
- Loops, Decision Conditions
- Each statement of your program
- Functions & Modules on an individual basis

Expected output



Process of White Box Testing

1. Input: Requirements, Functional specifications, design documents, source code.
2. Processing: Performing risk analysis to guide through the entire process.

3. Proper test planning: Designing test cases to cover the entire code. Execute rinse-repeat until error-free software is reached. Also, the results are communicated.
4. Output: Preparing the final report of the entire testing process.

White Testing is performed in 2 Steps

1. Tester should understand the code well
2. Tester should write some code for test cases and execute them

Features of White box Testing

1. Code coverage analysis: White box testing helps to analyze the code coverage of an application, which helps to identify the areas of the code that are not being tested.
2. Access to the source code: White box testing requires access to the application's source code, which makes it possible to test individual functions, methods, and modules.
3. Knowledge of programming languages: Testers performing white box testing must have knowledge of programming languages like Java, C++, Python, and PHP to understand the code structure and write tests.
4. Identifying logical errors: White box testing helps to identify logical errors in the code, such as infinite loops or incorrect conditional statements.
5. Integration testing: White box testing is useful for integration testing, as it allows testers to verify that the different components of an application are working together as expected.
6. Unit testing: White box testing is also used for unit testing, which involves testing individual units of code to ensure that they are working correctly.
7. Optimization of code: White box testing can help to optimize the code by identifying any performance issues, redundant code, or other areas that can be improved.
8. Security testing: White box testing can also be used for security testing, as it allows testers to identify any vulnerabilities in the application's code.
9. Verification of Design: It verifies that the software's internal design is implemented in accordance with the designated design documents.
10. Check for Accurate Code: It verifies that the code operates in accordance with the guidelines and specifications.
11. Identifying Coding Mistakes: It finds and fix programming flaws in your code, including syntactic and logical errors.
12. Path Examination: It ensures that each possible path of code execution is explored and test various iterations of the code.
13. Determining the Dead Code: It finds and remove any code that isn't used when the programme is running normally (dead code).

Advantages of White Box Testing

1. **Thorough Testing:** White box testing is thorough as the entire code and structures are tested.
2. **Code Optimization:** It results in the optimization of code removing errors and helps in removing extra lines of code.
3. **Early Detection of Defects:** It can start at an earlier stage as it doesn't require any interface as in the case of black box testing.
4. **Integration with SDLC:** White box testing can be easily started in Software Development Life Cycle.
5. **Detection of Complex Defects:** Testers can identify defects that cannot be detected through other testing techniques.
6. **Comprehensive Test Cases:** Testers can create more comprehensive and effective test cases that cover all code paths.
7. **Testers can ensure that the code meets coding standards and is optimized for performance.**

Disadvantages of White Box Testing

1. **Programming Knowledge and Source Code Access:** Testers need to have programming knowledge and access to the source code to perform tests.
2. **Overemphasis on Internal Workings:** Testers may focus too much on the internal workings of the software and may miss external issues.
3. **Bias in Testing:** Testers may have a biased view of the software since they are familiar with its internal workings.
4. **Test Case Overhead:** Redesigning code and rewriting code needs test cases to be written again.
5. **Dependency on Tester Expertise:** Testers are required to have in-depth knowledge of the code and programming language as opposed to black-box testing.
6. **Inability to Detect Missing Functionalities:** Missing functionalities cannot be detected as the code that exists is tested.
7. **Increased Production Errors:** High chances of errors in production.

White Box Techniques:

White Box Testing Techniques

One of the main benefits of white box testing is that it allows for testing every part of an application. To achieve complete code coverage, white box testing uses the following techniques:

1. Statement Coverage

It is one type of white box testing technique that ensures that all the statements of the source code are executed at least once. It covers all the paths, lines, and statements of a source code. It is used to design test box cases where it will find out the total number of executed statements out of the total statements present in the code.

*Statement coverage = (Number of executed statements / Total number of statements in source code) * 100*

Example 1:

```
Read A
Read B
if A > B
    Print "A is greater than B"
else
    Print "B is greater than A"
endif
```

Case 1:

```
If A = 7, B = 3
No of statements Executed = 5
Total statements = 7
Statement coverage = 5 / 7 * 100
                  = 71.00 %
```

Case 2:

```
If A = 4, B = 8
No of statements Executed = 6
Total statements = 7
Statement coverage = 6 / 7 * 100
                  = 85.20 %
```

2. Branch Coverage

Branch Coverage is a white box testing method in which every outcome from a code module (statement or loop) is tested. The purpose of branch coverage is to ensure that each decision condition from every branch is executed at least once. It helps to measure fractions of independent code segments and to find out sections having no branches.

For example, if the outcomes are binary, you need to test both True and False outcomes.

The formula to calculate Branch Coverage:

$$\text{Branch Coverage} = \frac{\text{Number of Executed Branches}}{\text{Total Number of Branches}}$$

there are two possible branches:

```
Demo(int a) {  
    If (a > 5)  
        a = a * 3  
    Print (a)  
}
```

1. When the condition ($a > 5$) evaluates to true.
2. When the condition ($a > 5$) evaluates to false.

To calculate the branch coverage:

- **Total branches:** There are 2 branches in the code.
- **Executed branches:** If both test cases (one where $a \leq 5$ and one where $a > 5$) have been executed during testing, then both branches have been executed.

If both branches are executed at least once during testing:

$$\text{Branch Coverage} = \left(\frac{2}{2} \right) \times 100 = 100\%$$

This means you would achieve 100% branch coverage if both conditions have been tested.

3. Condition Coverage

The software condition coverage testing is required to ensure that the program source code is working properly, and is able to satisfy the given requirements. It is observed that the traditional testing methodologies sometimes miss particular paths in the code, thereby some of the critical defects remain undetected. The software condition coverage testing takes care of these situations by methodically verifying all the conditions inside the decision points. Thus it improves the overall quality, and performance of the software.

The software condition coverage testing are performed by following the steps listed below –

Step 1 – Determine the decision points or conditional statements in the program source code like the if, else if, if, switch etc.

Step 2 – Every decision point may consist of more than one condition. These conditions are evaluated, and split into simpler modules to have an exhaustive testing.

Step 3 – Design test cases to include every possible result of all the conditions namely true, and false. It helps to cover every branch of code at the time of testing.

Step 4 – Execute the white box test cases, and evaluate the results. A coverage report is generated to measure the extent of verification of the conditions. It also describes the validated, and invalidated conditions, thereby it gives an idea if more fine tuning of the test cases are needed.

Advantages of Software Condition Coverage Testing

Example

```
Input X, Y, Z, and W
IF (X == 0 || Y == 0)
  THEN PRINT 100
  ELSE IF (Z == 0 && W == 0)
    THEN PRINT 200
  END IF
END
```

4. Loop Testing

Loop Testing is a type of software testing type that is performed to validate the loops. It is one of the types of Control Structure Testing. Loop testing is a white box testing technique and is used to test loops in the program.

Objectives of Loop Testing:

To fix the infinite loop repetition problem.

To know the performance.

To identify the loop initialization problems.

To determine the uninitialized variables.

Types of Loop Testing:

Loop testing is classified on the basis of the types of the loops:

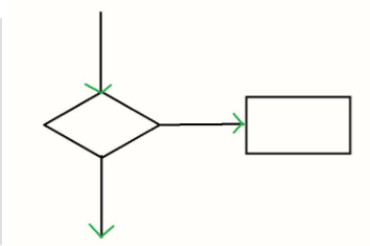
Simple Loop Testing:

Testing performed in a simple loop is known as Simple loop testing. Simple loop is basically a normal “for”, “while” or “do-while” in which a condition is given and loop runs and terminates according to true and false occurrence of the condition respectively. This type of testing is performed basically to test the condition of the loop whether the condition is sufficient to terminate loop after some point of time.

```

while(condition)
{
    statement(s);
}

```



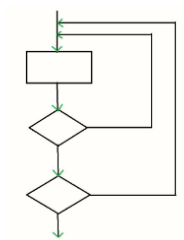
Nested Loop Testing:

Testing performed in a nested loop is known as Nested loop testing. Nested loop is basically one loop inside the another loop. In nested loop there can be finite number of loops inside a loop and there a nest is made. It may be either of any of three loops i.e., for, while or do-while.

```

while(condition 1)
{
    while(condition 2)
    {
        statement(s);
    }
}

```



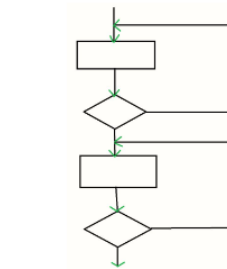
Concatenated Loop Testing

Testing performed in a concatenated loop is known as Concatenated loop testing. It is performed on the concatenated loops. Concatenated loops are loops after the loop. It is a series of loops. Difference between nested and concatenated is that in nested loop is inside the loop but here loop is after the loop.

```

while(condition 1)
{
    statement(s);
}
while(condition 2)
{
    statement(s);
}

```



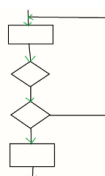
Unstructured Loop Testing:

Testing performed in an unstructured loop is known as Unstructured loop testing. Unstructured loop is the combination of nested and concatenated loops. It is basically a group of loops that are in no order.

```

while()
{
    for()
    {}
    while()
    {}
}

```



5. Basis Path Testing

In this technique, control flow graphs are made from code or flowchart and then Cyclomatic complexity is calculated which defines the number of independent paths so that the minimal number of test cases can be designed for each independent path.

Steps:

- Make the corresponding control flow graph
- Calculate the cyclomatic complexity
- Find the independent paths
- Design test cases corresponding to each independent path

There are three methods of computing Cyclomatic complexities.

Method 1: Total number of regions in the flow graph is a Cyclomatic complexity.

Method 2: The Cyclomatic complexity, $V(G)$ for a flow graph G can be defined as

$$V(G) = E - N + 2$$

Where: E is total number of edges in the flow graph.

N is the total number of nodes in the flow graph.

Method 3: The Cyclomatic complexity $V(G)$ for a flow graph G can be defined as

$$V(G) = P + 1$$

Where: P is the total number of predicate nodes contained in the flow G .

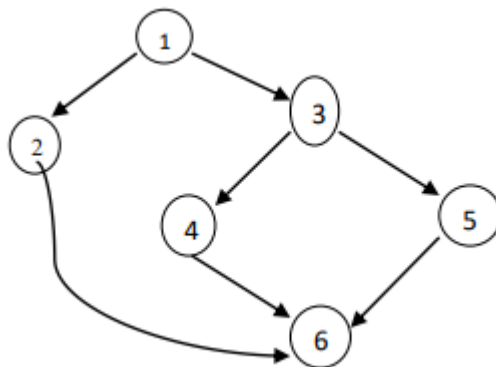
Let us understand computation of Cyclomatic complexity with the help of an example.

Consider following code fragment with line numbered

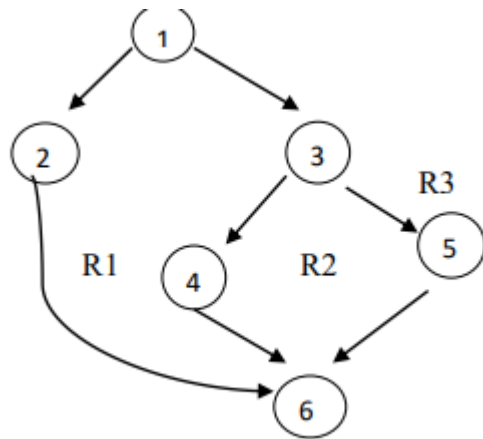
```
{  
1.If (a<b)  
2. F1();  
Else  
{  
3.If(a<c)  
4.F2();  
Else  
5.F3();  
}  
6.}
```

To compute Cyclomatic complexity we will follow these steps –

Step 1. Design flow graph for given code fragment.



Step 2. Compute region, Predicate (i.e decision nodes) edges and total nodes in the flow graph



There are 3 regions denoted by R1, R2 and R3.

Nodes 1 and 3 are predicate nodes because which branch to be followed is decided at these points.

Total edges = 7

Total nodes = 6

Step 3. Apply formulas in order to compute Cyclomatic complexity.

1) Cyclomatic complexity $V(G) = \text{Total number of regions} = 3$

2) Cyclomatic complexity $V(G) = E - N + 2$

Cyclomatic complexity $V(G) = 7 - 6 + 2 = 3$

3) Cyclomatic complexity $V(G) = P + 1$

$V(G) = 2 + 1 = 3$

Where P is predicate nodes (node 1 and node 2) are predicate nodes because from these nodes only the decision of which path is to be followed is taken.

Thus, Cyclomatic complexity is 3 for given code.

Independent Path

1-2-6

1-3-4-6

1-3-5-6

Gray Box Testing

Gray Box Testing is a software testing technique that is a combination of the Black Box Testing technique and the White Box Testing technique.

1. In the Black Box Testing technique, the tester is unaware of the internal structure of the item being tested and in White Box Testing the internal structure is known to the tester.
2. The internal structure is partially known in Gray Box Testing.
3. This includes access to internal data structures and algorithms to design the test cases.
4. Gray Box Testing is named so because the software program is like a semi-transparent or gray box inside which the tester can partially see.
5. It commonly focuses on context-specific errors related to web systems.
6. It is based on requirement test case generation because it has all the conditions presented before the program is tested.

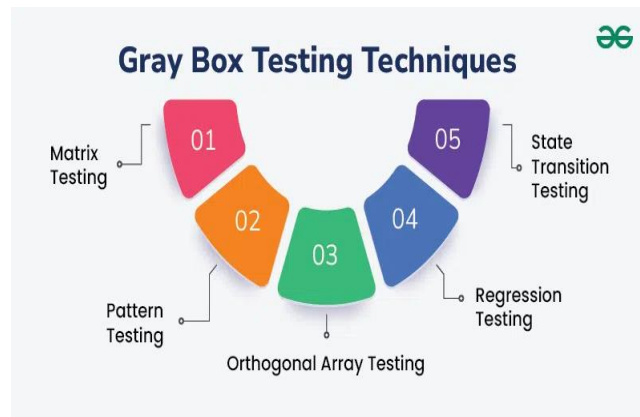


Objectives of Gray Box Testing

1. To provide combined advantages of both black box testing and white box testing.
2. To combine the input of developers as well as testers.
3. To improve overall product quality.
4. To reduce the overhead of long process of functional and non-functional testing.
5. To provide enough free time to developers to fix defects.
6. To test from the user point of view rather than a designer point of view.

Gray Box Testing Techniques

Here are the main techniques of the Gray Box Testing:



Gray Box Testing Techniques

1. Matrix Testing

In matrix testing technique, business and technical risks which are defined by the developers in software programs are examined. Developers define all the variables that exist in the program. Each of the variables has an inherent technical and business risk and can be used with varied frequencies during its life cycle.

2. Pattern Testing

To perform the pattern testing, previous defects are analysed. It determines the cause of the failure by looking into the code. Analysis template includes reasons for the defect. This helps test cases designed as they are proactive in finding other failures before hitting production.

3. Orthogonal Array Testing

It is mainly a black box testing technique. In orthogonal array testing, test data have n numbers of permutations and combinations. Orthogonal array testing is preferred when maximum coverage is required when there are very few test cases and test data is large. This is very helpful in testing complex applications.

4. Regression Testing

Regression testing is testing the software after every change in the software to make sure that the changes or the new functionalities are not affecting the existing functioning of the system. Regression testing is also carried out to ensure that fixing any defect has not impacted other functionality of the software.

5. State transition Testing

State transition testing is frequently applied to systems that display various states while they are being operated. Testers who have just a limited understanding of the internal states create test cases with the intention of making sure that state transitions are handled correctly.

6. Testing Decision Tables

Decision tables are a useful tool for organizing and condensing complicated business rules and reasoning. Decision tables are used by testers with limited understanding to generate test cases covering multiple combinations of input conditions and expected results.

7. Testing APIs

Even though the main code is not entirely known, gray box testing, also known as API (Application Programming Interface) testing, focuses on testing the system's exposed interfaces. The main goal of testing is to make sure the API accepts various input formats and operates as intended.

8. Data Flow Testing

Analysing the flow of data testing through the system forms the basis of data flow testing. Partial knowledge testers create test cases that examine the data's pathways throughout the application, assisting in the identification of possible problems with handling and processing the data.

Advantages of Gray Box Testing

1. Clarity of goals: Users and developers have clear goals while doing testing.
2. Done from user perspective: Gray box testing is mostly done by the user perspective.
3. High programming skills not required: Testers are not required to have high programming skills for this testing.
4. Improved product quality: Overall quality of the product is improved.
5. Defect fixing: In gray box testing, developers have more time for defect fixing.
6. Benefits of black box and white box testing: By doing gray box testing, benefits of both black box and white box testing is obtained.
7. Unbiased: Gray box testing is unbiased. It avoids conflicts between a tester and a developer.
8. Effective testing: Gray box testing is much more effective in integration testing.

Disadvantages of Gray Box Testing

1. Limited access to internal structure: Limited access to internal structure leads to limited access for code path traversal.
2. Source code not accessible: Because source code cannot be accessed, doing complete white box testing is not possible.
3. Not suitable for algorithm testing: Gray box testing is not suitable for algorithm testing.
4. Test cases difficult to design: Most of the test cases are difficult to design.

Black Box vs White Box vs Gray Box Testing

Here is a simple comparison of Black Box, White Box, and Gray Box testing, highlighting key aspects:

Aspect	Black Box Testing	White Box Testing	Gray Box Testing
Knowledge of Internal Code	Not required	Required	Partially required
Other Names	Functional testing, data-driven testing, closed box testing	Structural testing, clear box testing, code-based testing, transparent testing	Translucent testing
Approach	Trial and error, based on external functionality	Verification of internal coding, system boundaries, and data domains	Combination of both black box and white box approaches
Test Case Input Size	Largest	Smaller compared to Black Box	Smaller than both Black Box and White Box
Finding Hidden Errors	Difficult	Easier due to internal code access	Challenging, may be found at user level
Algorithm Testing	Not suitable	Well-suited and recommended	Not suitable
Time Consumption	Depends on functional specifications	High due to complex code analysis	Moderate, faster than White Box

